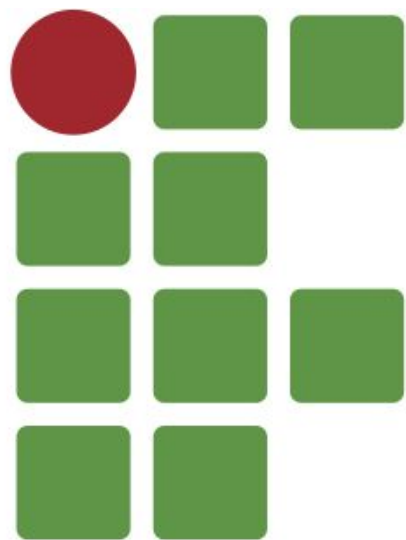




**INSTITUTO
FEDERAL**
Brasília

Algoritmos e Programação de Computadores TAI-35

Prof. MSc. Thiago Neves
IFB Campus Taguatinga
02/05/2026

Módulo 2

AULA 08

(02/04/2026)

Estrutura de repetição: do ... while
Uso de funções

main.c (ver 24) – do ... while(condição)

```
#include <stdio.h>
int main() {
    int i = 1;
    do {
        printf("%d\n", i);
        i++;
    } while (i <= 10);
    return 0;
}
```

Exercício 51 ~ 10min

Crie um programa que:

- Peça ao usuário para digitar números inteiros.
- Some os valores digitados.
- Pare quando o usuário digitar um número negativo.
- Mostre a soma final.

Exercício 52 ~ 10min

Crie um programa que:

- Exiba o seguinte menu:
 - 1 - Olá
 - 2 - Data fictícia
 - 3 - Sair
- Execute a opção escolhida.
- O menu deve continuar aparecendo até o usuário escolher sair.
- Use do...while.

Exercício 53 ~ 10min

Crie um programa que:

- Peça ao usuário uma nota entre 0 e 10.
- Se o valor for inválido, peça novamente.
- Continue solicitando até que seja digitado um valor válido.
- Use do...while.

Exercício 54 ~ 10min

Crie um programa que:

- Peça um número ao usuário.
- Mostre a tabuada desse número de 1 a 10.
- Ao final, pergunte se o usuário deseja calcular outra tabuada (S/N).
- Repita enquanto a resposta for 'S'.

Forma Geral de uma Função

```
tipo nome_da_função(lista da parâmetros) {  
    // corpo da função  
    retorno;  
}
```

Exemplo:

```
int soma(int a, int b) {  
    return a+b;  
}
```

Funções em C

1. Cada função é um bloco discreto de código, ou seja, um código de uma função é privativo àquela função.
2. Uma variável definida em uma função é criada ao iniciar a função e destruída (desalocada) ao sair.
3. Os parâmetros de funções podem ser passados por valor (cópia) ou por referência (endereços de memória – uso de ponteiros).

main.c (ver 25) - Uso de funções void

```
#include <stdio.h>
```

```
void imprime() {  
    printf("Ola mundo!\n");  
}
```

```
int main() {  
    imprime();  
    imprime();  
    return 0;  
}
```

main.c (ver 26) - Funções com argumentos e retornos

```
#include <stdio.h>

int potencia(int a, int b) {
    int resultado = a;
    for (int i = 1; i < b; i++) {
        resultado *= a;
    }
    return resultado;
}

int main() {
    int a = 1, b = 8;
    printf("<%d> elevado a potencia <%d> e: %d", a, b, potencia(a, b));
    return 0;
}
```

Exercício 55 ~ 10min

Modifique o código anterior (main.c ver 26) para tratar:

- `base = 0;`
- `base < 0;`
- `expoente = 0;`
- `expoente < 0.`

main.c (ver 27) - Passagem de strings como argumento de funções

```
#include <stdio.h>
```

```
void imprimir(char nome[20]) {  
    printf(nome);  
}
```

```
int main() {  
    char frase1[] = "O rato roeu a roupa do rei de roma!\n";  
    char frase2[] = "Quem tem boca, vai a roma!\n";  
    imprimir(frase1);  
    imprimir(frase2);  
    return 0;  
}
```

main.c (ver 28) - Escopo de variáveis em funções

```
#include <stdio.h>

void play(int nro) {
    nro += 1;
    printf("play: %d\n", nro);
}

int main() {
    int nro = 10;
    printf("main: %d\n", nro);
    nro += 1;
    printf("main: %d\n", nro);
    nro += 1;
    printf("main: %d\n", nro);
    nro += 1;
    play(nro);
    printf("main: %d\n", nro);
    return 0;
}
```

main.c (ver 29) - Funções com retorno

```
#include <stdio.h>
```

```
int sqr(int x) {  
    return x*x;  
}
```

```
int main() {  
    int x=11;  
    printf("%d %d \n", x, sqr(x));  
    return 0;  
}
```

main.c (ver 30) - Variáveis globais

```
#include <stdio.h>
int contador = 0; // Variável global
void incrementar() {
    contador++;
}
void mostrar() {
    printf("Valor do contador: %d\n", contador);
}
int main() {
    mostrar();
    incrementar();
    incrementar();
    mostrar();
    return 0;
}
```

Exercício 56 ~ 10min

Crie:

- `int somaDigitos(int n);`

A função deve retornar a soma dos dígitos de um número inteiro positivo.

Exemplo: $358 \rightarrow 3 + 5 + 8 = 16$.

Exercício 57 ~ 10min

Crie as funções:

- `float soma(float a, float b);`
- `float subtracao(float a, float b);`
- `float multiplicacao(float a, float b);`
- `float divisao(float a, float b);`

No main, implemente um menu que permita ao usuário escolher a operação.

O programa só deve encerrar quando o usuário escolher sair.

Exercício 58 ~ 10min

Crie a função:

- `int palindromo(int n);`

A função deve verificar se o número lido é igual ao seu inverso.
Exemplo: 121 → verdadeiro.

Exercício 59 ~ 10min

Crie:

- `float calcularSerie(int n);`

Calcular:

$$S = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$$

Exercício 60 ~ 10min

Crie:

- `int dataValida(int dia, int mes, int ano);`

A função deve:

- Considerar ano bissexto (divisível por 4)
- Validar meses com 30 e 31 dias
- Retornar 1 se válida, 0 se inválida

AULA 09

(23/04/2026)

E/S com Arquivo - Parte 1

C ANSI - Streams

Um stream em C é uma abstração de fluxo de dados usada para entrada e saída. Ele permite que o programa trate teclado, tela, arquivos e outros dispositivos de maneira uniforme.

Tipos de streams:

- `stdin` → entrada padrão.
- `stdout` → saída padrão.
- `stderr` → saída de erro.
- `FILE *` → stream associado a arquivo.

C ANSI - Arquivos

Em C, um **arquivo** pode ser qualquer coisa, desde um arquivo em disco até um terminal ou impressora. Você associa uma stream com um arquivo específico realizando uma operação de abertura. Uma vez o arquivo aberto, informações podem ser trocadas entre ele e seu programa.

Nem todo arquivo suporta acesso aleatório. Ex: teclado.

Se o arquivo suportar acesso aleatório, abrir esse arquivo também inicializa o indicador de posição do arquivo para o início do arquivo. Quando cada caractere é lido ou escrito, o indicador é incrementado.

C ANSI - Funções de manipulação de arquivos

Nome	Função
fopen()	Abre um arquivo
fclose()	Fecha um arquivo
putc()	Escreve um caractere em um arquivo
fputc()	O mesmo que putc()
getc()	Lê um caractere de um arquivo
fgetc()	O mesmo que getc()
fseek()	Posiciona o arquivo em um byte específico
fprintf()	É para um arquivo o que printf() é para o console
fscanf()	É para um arquivo o que scanf() é para o console
feof()	Devolve verdadeiro se o fim de arquivo for atingido
ferror()	Devolve verdadeiro se ocorreu um erro
rewind()	Recoloca o indicador de posição de arquivo no início do arquivo
remove()	Apaga um arquivo
fflush()	Descarrega um arquivo

C ANSI - Macros

- **NULL**: ponteiro nulo.
- **EOF**: indica o final do arquivo (-1).
- **FOPEN_MAX**: número de arquivos que podem ser abertos simultaneamente (≥ 8).
- **SEEK_SET**, **SEEK_CUR**, **SEEK_END**: usadas para acesso aleatório em arquivos.

Função fopen()

```
#include <stdio.h>
```

```
FILE *fopen(const char *filename, const char *mode);
```

Modo	Significado
r	Abre um arquivo-texto para leitura
w	Cria um arquivo-texto para escrita
a	Anexa a um arquivo-texto
rb	Abre um arquivo binário para leitura
wb	Cria um arquivo binário para escrita
ab	Anexa a um arquivo binário
r+	Abre um arquivo-texto para leitura/escrita
w+	Cria um arquivo-texto para leitura/escrita
a+	Anexa ou cria um arquivo-texto para leitura/escrita
r+b	Abre um arquivo binário para leitura/escrita
w+b	Cria um arquivo binário para leitura/escrita
a+b	Anexa a um arquivo binário para leitura/escrita

main.c (ver 31) - Abrindo um arquivo para leitura

```
#include <stdio.h>
```

```
int main() {  
    FILE *f;  
    f = fopen("dados.txt", "r");  
    if (f == NULL) {  
        printf("Erro ao abrir o arquivo!");  
    }  
  
    fclose(f);  
    return 0;  
}
```

Função fprintf()

```
#include <stdio.h>
```

```
int fprintf(FILE *stream, const char *format-string, argument-list);
```

A função fprintf() formata e grava uma série de caracteres e valores na saída stream. A função fprintf() converte cada entrada em argument-list, se houver, e grava no fluxo de acordo com a especificação do formato correspondente na format-string.

Função putc()

```
#include <stdio.h>
```

```
int putc(int c, FILE *stream);
```

A função putc() converte c em unsigned char e, em seguida, grava c na saída stream na posição atual.

main.c (ver 32) - Escrita com fprintf

```
#include <stdio.h>

int count [10] = {1, 5, 11, 11, 0, 3, 5, 6, 8, 10};

int main() {
    int i,j;
    FILE *f;
    f = fopen("dados.txt", "w");
    for (i=0; i < 10; i++) {
        for (j = 0; j < count[i]; j++)
            fprintf(f, "*");

        fprintf(f, "\n");
    }
    fclose (f);
}
```

OBS: Este exemplo envia uma linha de asteriscos para cada inteiro na matriz count para o arquivo. O número de asteriscos impressos em cada linha corresponde a um número inteiro na matriz.

main.c (ver 33) - Escrita com putc

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    FILE *f;
    f = fopen("saida.txt", "w");
    if (f == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }
    putc('O', f);
    putc('l', f);
    putc('a', f);
    putc('\n', f);
    fclose(f);
    return 0;
}
```

Exercício 61 ~ 10min

Escreva um programa em C que crie ou sobrescreva o arquivo `saida.txt` e grave nele a frase:

“Olá, arquivo!”

Usando as funções `fprintf` ou `putc`.

Exercício 62 ~ 10min

Escreva um programa em C que leia do teclado o nome e a idade de uma pessoa e grave essas informações no arquivo `pessoa.txt` no formato:

Nome: Ana

Idade: 25

Use `fprintf` para a gravação.

Exercício 63 ~ 10min

Escreva um programa em C que abra o arquivo log.txt em modo de acréscimo (a) e adicione a frase:

Programa executado com sucesso!

Ao final do arquivo, seguida de quebra de linha, usando fprintf.

Rode o executável várias vezes para verificar o funcionamento do acréscimo ao final do arquivo.

Exercício 64 ~ 10min

Escreva um programa em C que leia do teclado 4 notas de um aluno, calcule a média e grave no arquivo boletim.txt as notas e a média no formato:

Nota 1: 8.0

Nota 2: 7.5

Nota 3: 9.0

Nota 4: 6.5

Média: 7.75

Use fprintf.

Função fscanf

```
#include <stdio.h>
```

```
int fscanf (FILE *stream, const char *format-string, argument-list);
```

A função fscanf() lê dados da posição atual do stream especificado nos locais que são fornecidos pelas entradas em argument-list, se houver. Cada entrada em argument-list deve ser um ponteiro para uma variável com um tipo que corresponde a um especificador de tipo em format-string.

main.c (ver 34) - Leitura com fscanf

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    FILE *f;
    char nome[50];
    int idade;
    f = fopen("dados.txt", "r");
    if (f == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }
    while (fscanf(f, "%49s %d", nome, &idade) == 2) {
        printf("Nome: %s | Idade: %d\n", nome, idade);
    }
    fclose(f);
    return 0;
}
```

Função fgets

```
#include <stdio.h>
```

```
char *fgets (char *string, int n, FILE *stream);
```

A função fgets() lê caracteres da posição atual do fluxo até e incluindo o primeiro caractere de nova linha (\n), até o final do fluxo ou até que o número de caracteres lidos seja igual a n-1, o que ocorrer primeiro.

A função fgets() armazena o resultado em string e inclui um caractere nulo (\0) no término da cadeia. A sequência inclui o caractere de nova linha, se for lido. Se n for igual a 1, a sequência estará vazia.

main.c (ver 35) - Leitura com fgets

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    FILE *f;
    char linha[100];
    f = fopen("dados.txt", "r");
    if (f == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }
    while (fgets(linha, sizeof(linha), f) != NULL) {
        printf("%s", linha);
    }
    fclose(f);
    return 0;
}
```

Exercício 65 ~ 10min

Escreva um programa em C que leia, usando fgets, o nome completo de um usuário e exiba a mensagem:

Bem-vindo(a), <NOME>!

Exemplo de arquivo de leitura (dados.txt):

Ana Maria da Silva

Exercício 66 ~ 10min

Escreva um programa em C que leia, usando scanf, dois números inteiros e mostre:

- soma
- subtração
- multiplicação
- divisão inteira

Exemplo de arquivo de leitura (dados.txt):

10 5

Exercício 67 ~ 10min

Escreva um programa em C que leia:

- o nome completo de um aluno com fgets
- duas notas com scanf

Depois, mostre o nome do aluno e a média das notas.

Exemplo de arquivo de leitura (dados.txt):

Carlos Eduardo Lima

7.5

8.0

Exercício 68 ~ 10min

Escreva um programa em C que leia, de um arquivo chamado alunos.txt, os dados de 5 alunos. Para cada aluno, o arquivo conterá:

nome completo

nota1nota2 nota3

Exemplo de arquivo de leitura (dados.txt):

Ana Beatriz

7.5 8.0 6.5

Carlos Eduardo

5.0 4.5 6.0

Mariana Silva

9.0 8.5 9.5

Joao Pedro

6.0 5.5 6.0

Fernanda Lima

3.0 4.0 2.5

Exercício 69 ~ 10min

Escreva um programa em C que leia de um arquivo chamado estoque.txt os dados de vários produtos. Para cada produto, o arquivo conterá:

código

nome do produto

quantidade em estoque

preço unitário

Exemplo de arquivo de leitura (estoque.txt):

101

Caderno Universitario

25 18.50

102

Caneta Azul

8 2.75

103

Mochila Executiva

4 129.90

Exercício 70 ~ 10min

Escreva um programa em C que leia de um arquivo chamado clima.txt os dados de temperatura de 7 dias. Para cada dia, o arquivo conterá:

- nome do dia da semana
- temperatura mínima
- temperatura máxima
- índice de chuva em milímetros

Exemplo de arquivo de leitura (clima.txt):

Segunda 18.5 29.0 0.0

Terça 19.0 30.5 2.5

Quarta 17.0 28.0 10.0

Quinta 16.5 27.5 5.0

Sexta 20.0 31.0 0.0

Sábado 21.0 32.5 1.0

Domingo 19.5 30.0 15.0

AULA 10

(30/4/2026)

E/S com Arquivo - Parte 2

Função getc

```
#include <stdio.h>
```

```
int getc(FILE *stream);
```

A função `getc()` lê um único caractere da posição atual do fluxo e avança a posição do fluxo para o próximo caractere.

main.c (ver 36) - Leitura com fgetc

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    FILE *arquivo;
    int c;
    arquivo = fopen("dados.txt", "r");
    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }
    while ((c = fgetc(arquivo)) != EOF) {
        putchar(c);
    }
    if (feof(arquivo)) {
        printf("\nFim do arquivo alcançado.\n");
    }
    fclose(arquivo);
    return 0;
}
```

Função remove

```
#include <stdio.h>
```

```
int remove(const char *filename);
```

A função remove() exclui o arquivo especificado pelo filename.

Exercício 71 ~ 10min

Crie um programa em C que:

- abra um arquivo chamado texto.txt em modo leitura;
- leia todo o conteúdo do arquivo usando getc;
- mostre o conteúdo na tela caractere por caractere;
- conte quantos caracteres existem no arquivo;
- ao final, pergunte ao usuário se deseja apagar o arquivo;
- caso a resposta seja 'S', remova o arquivo usando remove.

Função fseek

```
#include <stdio.h>
```

```
int fseek(FILE *stream, long int offset, int origin);
```

A função `fseek()` muda a posição do arquivo atual associada ao `stream` para um novo local dentro do arquivo. A próxima operação no `stream` ocorrerá no novo local. Em um fluxo aberto para atualização, a próxima operação pode ser uma operação de leitura ou de gravação.

Macros:

SEEK_SET: Início do arquivo.

SEEK_CUR: Posição atual do ponteiro do arquivo.

SEEK_END: fim de arquivo.

main.c (ver 37) - Ir para o início do arquivo

```
#include <stdio.h>
```

```
int main() {  
    FILE *f = fopen("dados.txt", "r");  
    if (f == NULL) {  
        printf("Erro ao abrir o arquivo.\n");  
        return 1;  
    }  
    fseek(f, 0, SEEK_SET);    // leva o cursor para o byte 0, ou seja, para o começo do arquivo.  
    printf("Cursor movido para o início do arquivo.\n");  
  
    fclose(f);  
    return 0;  
}
```

main.c (ver 38) - Ler a partir de uma posição específica

```
#include <stdio.h>
```

```
int main() {  
    FILE *f = fopen("dados.txt", "r");  
    char c;  
    if (f == NULL) {  
        printf("Erro ao abrir o arquivo.\n");  
        return 1;  
    }  
    fseek(f, 3, SEEK_SET);          // posiciona no 4º caractere.  
    c = fgetc(f);  
    printf("Caractere lido: %c\n", c);  
    fclose(f);  
    return 0;  
}
```

main.c (ver 39) - Voltar alguns bytes a partir da posição atual

```
#include <stdio.h>

int main() {
    FILE *f = fopen("dados.txt", "r");
    char c;
    if (f == NULL) {
        printf("Erro ao abrir o arquivo.\n");
        return 1;
    }
    fgetc(f); // A
    fgetc(f); // B
    fgetc(f); // C
    fseek(f, -1, SEEK_CUR); // volta 1 byte.
    c = fgetc(f);
    printf("Caractere lido: %c\n", c);
    fclose(f);
    return 0;
}
```

main.c (ver 40) - Descobrir o tamanho do arquivo

```
#include <stdio.h>
```

```
int main() {  
    FILE *f = fopen("dados.txt", "r");  
    long tamanho;  
    if (f == NULL) {  
        printf("Erro ao abrir o arquivo.\n");  
        return 1;  
    }  
    fseek(f, 0, SEEK_END);  
    tamanho = ftell(f); // informa a posição atual. Essa posição corresponde ao tamanho do arquivo em bytes.  
    printf("Tamanho do arquivo: %ld bytes\n", tamanho);  
    fclose(f);  
    return 0;  
}
```

main.c (ver 41) - Gravar em posição específica

```
#include <stdio.h>
```

```
int main() {  
    FILE *f = fopen("dados.txt", "r+");  
    if (f == NULL) {  
        printf("Erro ao abrir o arquivo.\n");  
        return 1;  
    }  
  
    fseek(f, 2, SEEK_SET);    // o cursor vai para a posição 2.  
    fputc('X', f);           // sobrescreve o caractere daquela posição.  
    fclose(f);  
    return 0;  
}
```

Exercício 72 ~ 10min

Escreva um programa em C que:

- Leia o conteúdo de origem.txt usando getc e grave uma cópia idêntica em destino.txt usando putc.
- Conte quantas vogais e consoantes existe no arquivo origem.txt e imprima na tela do usuário.

Exercício 73 ~ 10min

Escreva um programa em C que:

- Leia 10 números inteiros do arquivo `numeros.txt` usando `fscanf`, calcule a soma de todos e grave o resultado em `soma.txt` usando `fprintf`.

Exercício 74 ~ 10min

Escreva um programa em C que:

- Leia de alunos.txt o nome e duas notas de vários alunos usando fscanf, calcule a média de cada um e grave em relatorio.txt o nome, a média e a situação final (APROVADO ou REPROVADO) usando fprintf.

Exercício 75 ~ 10min

Escreva um programa em C que:

- Abra texto.txt, use fseek para posicionar o cursor no último caractere do arquivo e mostre esse caractere na tela usando getc.
- Determine o tamanho do arquivo em bytes e imprima na tela.

OBS: EOF não é um caractere armazenado no arquivo. Ele é um valor especial retornado por funções como getc, fgetc ou scanf para indicar que não há mais dados para ler.

Exercício 76 ~ 10min

Escreva um programa em C que abra dados.txt em modo de leitura e escrita, use fseek para ir até a terceira posição do arquivo e substitua o caractere existente por X usando putc.

Exercício 77 ~ 10min

Escreva um programa em C que leia um arquivo texto chamado entrada.txt e gere outro arquivo chamado compactado.txt usando uma compactação simples por repetição de caracteres.

Exemplo:

entrada: AAABBBCCDAA

saída: A3B3C2D1A2

O programa deve ler o arquivo original e escrever o conteúdo compactado no arquivo de saída.

Exercício 78 ~ 10min

Escreva um programa em C que leia de um arquivo chamado votos.txt uma sequência de votos de uma eleição, em que cada linha contém:

número do candidato

Considere também votos:

0 para branco

números negativos para nulo

O programa deve:

- contabilizar os votos de cada candidato;
- calcular total de votos válidos, brancos e nulos;
- identificar o candidato vencedor;
- gravar um relatório completo em resultado_eleicao.txt.

Exercício 79 (Avançado)

Crie um programa em C que leia um arquivo chamado ranking.txt, contendo jogadores e suas pontuações no seguinte formato:

Ana 150

Carlos 220

Beatriz 180

Daniel 90

O programa deve:

- Ler todos os jogadores do arquivo;
- Solicitar ao usuário o nome de um jogador e uma nova pontuação;
- Se o jogador já existir, atualizar sua pontuação somando a nova pontuação à anterior;
- Se o jogador não existir, adicioná-lo ao ranking;
- Ordenar os jogadores em ordem decrescente de pontuação;
- Gravar o ranking atualizado em ranking.txt.

Exercício 80 (Avançado)

Crie um programa que leia dois arquivos:

gabarito.txt

A
C
D
B
E

respostas.txt

2024001 A C D B E
2024002 A B D B A
2024003 C C D E E

Cada linha de respostas.txt contém a matrícula do aluno seguida de suas respostas.
O programa deve:

- Ler o gabarito;
- Ler as respostas de cada aluno;
- Calcular a nota de cada aluno;
- Considerar 1 ponto por questão correta;
- Gerar um arquivo resultado.txt no seguinte formato:

MATRICULA NOTA
2024001 5
2024002 3
2024003 3

AULA 11

(07/5/2026)

Header files

Uso de structs

Leitura/Escrita de structs

Header files

Em C, header files são arquivos de cabeçalho, geralmente com extensão “.h”, usados para declarar informações que serão compartilhadas entre vários arquivos .c.

Os header files servem principalmente para guardar declarações, como:

```
int somar(int a, int b);
```

Essa linha informa ao compilador que existe uma função chamada somar, que recebe dois inteiros e retorna um inteiro. A implementação da função pode estar em outro arquivo .c.

main.c (ver 42) - Uso de Header files

matematica.h

```
#ifndef MATEMATICA_H
#define MATEMATICA_H
int somar(int a, int b);
int subtrair(int a, int b);
#endif
```

matematica.c

```
#include "matematica.h"
int somar(int a, int b) {
    return a + b;
}
int subtrair(int a, int b) {
    return a - b;
}
```

main.c

```
#include <stdio.h>
#include "matematica.h"

int main() {
    int resultado = somar(10, 5);

    printf("Resultado: %d\n", resultado);

    return 0;
}
```

.h -> Contém declarações.

.c -> Contém implementações

Por que usar header files?

1. Organizar melhor o código;
2. Reutilizar funções em vários arquivos;
3. Separar declaração de implementação;
4. Facilitar manutenção;
5. Permitir projetos maiores com vários arquivos .c.

Exercício 81 ~ 10min

Crie um programa em C dividido em três arquivos:

- calculadora.h
- calculadora.c
- main.c

O arquivo calculadora.h deve conter os protótipos das seguintes funções:

- int somar(int a, int b);
- int subtrair(int a, int b);
- int multiplicar(int a, int b);

float dividir(int a, int b)

O arquivo calculadora.c deve implementar essas funções.

O arquivo main.c deve incluir o header calculadora.h, ler dois números inteiros do usuário e mostrar o resultado das quatro operações.

Structs em C

Em C, struct é uma forma de criar um tipo de dado composto, reunindo várias informações diferentes em uma única estrutura.

Enquanto um int guarda um número inteiro e um float guarda um número decimal, uma struct pode guardar vários dados relacionados ao mesmo objeto.

```
struct Aluno {  
    char nome[50];  
    int idade;  
    float nota;  
};
```

```
struct Aluno aluno1;  
struct Aluno aluno2;
```

main.c (ver 43) - Declarando structs

```
#include <stdio.h>
#include <string.h>
struct Aluno {
    char nome[50];
    int idade;
    float nota;
};
int main() {
    struct Aluno aluno1;
    strcpy(aluno1.nome, "Ana");
    aluno1.idade = 20;
    aluno1.nota = 8.5;
    printf("Nome: %s\n", aluno1.nome);
    printf("Idade: %d\n", aluno1.idade);
    printf("Nota: %.2f\n", aluno1.nota);
    return 0;
}
```

Função memset

```
#include <string.h>
```

```
void *memset(void *dest, int c, size_t count);
```

A função `memset()` configura os primeiros `count` bytes de `dest` para o valor `c`.

Função memcpy

```
#include <string.h>
```

```
void *memcpy(void *dest, const void *src, size_t count);
```

A função memcpy() copia count bytes de src para dest. O comportamento é indefinido se a cópia ocorrer entre os objetos que se sobrepõem.

main.c (ver 44)

```
#include <stdio.h>
#include <string.h>
struct Aluno {
    char nome[50];
    int idade;
    float nota;
};
int main() {
    struct Aluno aluno1, aluno2;
    memset(&aluno1, 0, sizeof(struct Aluno));
    memset(&aluno2, 0, sizeof(struct Aluno));
    strcpy(aluno1.nome, "Ana");
    aluno1.idade = 20;
    aluno1.nota = 8.5;
    memcpy(&aluno2, &aluno1, sizeof(struct Aluno) );
    printf("Nome: %s\n", aluno2.nome);
    printf("Idade: %d\n", aluno2.idade);
    printf("Nota: %.2f\n", aluno2.nota);
    return 0;
}
```

Exercício 82 ~ 10min

Declare uma struct chamada Livro com os seguintes campos:

título;

autor;

ano de publicação;

quantidade de páginas;

preço.

Os campos titulo e autor devem permitir armazenar textos com até 100 caracteres.

Exercício 83 ~ 10min

Declare uma struct chamada Produto para representar um item de estoque.

Ela deve conter:

- código do produto;
- nome do produto;
- quantidade em estoque;
- preço unitário;
- categoria.

A categoria deve ser armazenada como texto com até 30 caracteres.

Exercício 84 ~ 10min

Declare duas structs:

Uma struct chamada Endereco, contendo:

- rua;
- número;
- bairro;
- cidade;
- estado.

Uma struct chamada Cliente, contendo:

- nome;
- CPF;
- telefone;
- um endereço do tipo Endereco.

Considere:

- nome com até 80 caracteres;
- CPF com até 14 caracteres;
- telefone com até 15 caracteres;
- rua, bairro e cidade com até 60 caracteres;
- estado com até 2 caracteres.

Exercício 85 ~ 10min

Em C, crie um programa que utilize uma struct dentro de outra struct para cadastrar os dados de um aluno.

```
typedef struct {  
    char rua[80];  
    int numero;  
    char bairro[50];  
    char cidade[50];  
    char estado[3];  
} Endereco;
```

```
typedef struct {  
    char nome[80];  
    int matricula;  
    int idade;  
    float media;  
    Endereco endereco;  
} Aluno;
```

Função fread

```
#include <stdio.h>
```

```
size_t fread(void *buffer, size_t size, size_t count, FILE *stream);
```

A função fread() lê até count itens de size comprimento da entrada stream e os armazena no buffer fornecido. A posição no arquivo aumenta pelo número de bytes lidos.

Função fwrite

```
#include <stdio.h>
```

```
size_t fwrite(const void *buffer, size_t size, size_t count, FILE  
*stream);
```

A função fwrite() grava até count itens, cada um dos size bytes de comprimento, de buffer para o stream de saída.

main.c (ver 45) - Gravando e lendo um inteiro em arquivo binário

```
#include <stdio.h>

int main() {
    FILE *arquivo;
    int numero = 100;
    int numeroLido;
    arquivo = fopen("dados.bin", "wb"); // Gravando o número no arquivo
    if (arquivo == NULL) return 1;
    fwrite(&numero, sizeof(int), 1, arquivo);
    fclose(arquivo);

    arquivo = fopen("dados.bin", "rb"); // Lendo o número do arquivo
    if (arquivo == NULL) return 1;
    fread(&numeroLido, sizeof(int), 1, arquivo);
    fclose(arquivo);
    printf("Numero lido do arquivo: %d\n", numeroLido);
    return 0;
}
```

main.c (ver 46) - Gravando e lendo uma struct em arquivo binário

```
#include <stdio.h>

#include <string.h>

typedef struct {
    char nome[50]; int idade; float media;
} Aluno;

int main() {
    FILE *arquivo;

    Aluno aluno, alunoLido;

    strcpy(aluno.nome, "Ana Souza");

    aluno.idade = 20;

    aluno.media = 8.5;

    arquivo = fopen("aluno.bin", "wb"); // Gravando a struct no arquivo

    if (arquivo == NULL) return 1;

    fwrite(&aluno, sizeof(Aluno), 1, arquivo);

    fclose(arquivo);

    arquivo = fopen("aluno.bin", "rb"); // Lendo a struct do arquivo

    if (arquivo == NULL) return 1;

    fread(&alunoLido, sizeof(Aluno), 1, arquivo);

    fclose(arquivo);

    printf("Nome: %s\nIdade: %d\nMedia: %.2f\n", alunoLido.nome, alunoLido.idade, alunoLido.media);

    return 0;
}
```

Exercício 86 ~ 10min

Crie um programa em C que armazene dados de alunos em um arquivo binário chamado alunos.dat.

Use a seguinte struct:

```
typedef struct {  
    int matricula;  
    char nome[80];  
    float nota1;  
    float nota2;  
    float media;  
} Aluno;
```

O programa deve:

- Cadastrar vários alunos;
- Calcular automaticamente a média de cada aluno;
- Gravar cada aluno no arquivo usando fwrite;
- Ler todos os alunos gravados usando fread;
- Exibir os dados no formato:

Matrícula: 2024001 | Nome: Ana Souza | Média: 8.50

Exercício 87 ~ 10min

Crie um programa que leia e atualize produtos armazenados em um arquivo binário chamado produtos.dat.

Use a seguinte struct:

```
typedef struct {  
    int codigo;  
    char nome[60];  
    int quantidade;  
    float preco;  
} Produto;
```

O programa deve:

- Cadastrar produtos no arquivo usando fwrite;
- Solicitar ao usuário o código de um produto;
- Procurar esse produto no arquivo usando fread;
- Se encontrado, solicitar uma nova quantidade e um novo preço;
- Voltar a posição correta do arquivo usando fseek;
- Sobrescrever apenas o registro encontrado usando fwrite;
- Exibir todos os produtos após a atualização.

Exercício 88 ~ 10min

Crie um programa em C que gerencie clientes em um arquivo binário chamado clientes.dat.

Use a seguinte struct:

```
typedef struct {  
    int codigo;  
    char nome[80];  
    char cpf[15];  
    int ativo;  
} Cliente;
```

O campo ativo deve indicar:

1 = cliente ativo
0 = cliente removido

O programa deve:

- Cadastrar clientes no arquivo;
- Listar apenas os clientes ativos;
- Solicitar o código de um cliente para remoção;
- Localizar o cliente no arquivo;
- Alterar o campo ativo para 0;
- Usar fseek e fwrite para atualizar o registro no próprio arquivo;
- Confirmar se o cliente foi removido logicamente.

Exercício 89 ~ 10min

Crie um programa que grave e leia vendas em um arquivo binário chamado vendas.dat.

Use a seguinte struct:

```
typedef struct {  
    int codigoVenda;  
    char produto[60];  
    int quantidade;  
    float valorUnitario;  
} Venda;
```

O programa deve:

- Cadastrar várias vendas usando fwrite;
- Ler todas as vendas usando fread;
- Calcular o valor total de cada venda:
total = quantidade * valorUnitario;
- Exibir cada venda na tela;
- Calcular o faturamento total da loja;
- Identificar a venda de maior valor total.

Exemplo de saída:

Venda: 101 | Produto: Teclado | Total: R\$ 250.00
Venda: 102 | Produto: Monitor | Total: R\$ 900.00

Faturamento total: R\$ 1150.00
Maior venda: 102 - Monitor - R\$ 900.00

Exercício 90 ~ 10min

Crie um programa em C que armazene contas bancárias em um arquivo binário chamado contas.dat.

Use a seguinte struct:

```
typedef struct {  
    int numero;  
    char titular[80];  
    float saldo;  
} Conta;
```

O programa deve apresentar um menu com as opções:

- 1 - Cadastrar conta
- 2 - Listar contas
- 3 - Depositar
- 4 - Sacar
- 5 - Sair

O programa deve:

- Cadastrar contas usando fwrite;
- Listar todas as contas usando fread;
- Procurar uma conta pelo número;
- Realizar depósito, atualizando o saldo no arquivo;
- Realizar saque, atualizando o saldo no arquivo;
- Impedir saque maior que o saldo disponível;
- Usar fseek para retornar à posição correta do registro antes de sobrescrevê-lo.

AULA 12

(14/5/2026)

Matrizes em C

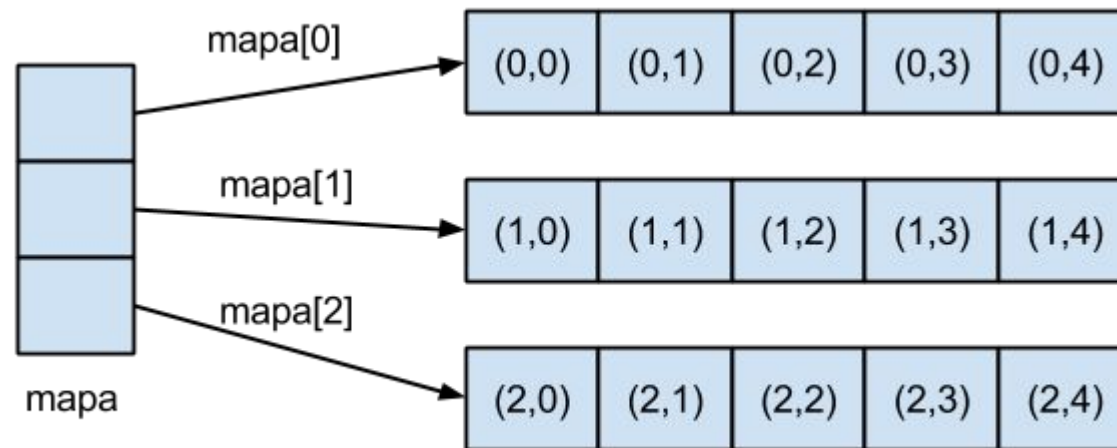
Representação de Matrizes - Parte 1

Um array/vetor é simplesmente um ponteiro que aponta para uma posição da memória que tem espaços vazios para guardar as N posições declaradas.

Uma matriz é uma abstração ainda maior. Ela é um ponteiro que aponta para uma lista de arrays. Como arrays são ponteiros, então uma matriz é um ponteiro que aponta para uma lista de ponteiros.

Representação de Matrizes - Parte 2

No exemplo, mapa é um ponteiro que aponta para outros ponteiros. Isso significa que `mapa[0]` é um ponteiro que aponta para um array, `mapa[1]` é um ponteiro que aponta para outro array, e assim por diante.



Representação de Matrizes - Parte 3

Em C, nada nos impede de declararmos uma matriz e depois declarar ponteiros que apontam para cada um dos vetores dentro dessa matriz. Veja o código abaixo. `linha0[0]` é igual a `numeros[0][0]`, afinal tanto `linha0` quanto `numeros[0]` apontam para o mesmo endereço de memória.

```
int numeros[5][10];
```

```
int* linha0 = numeros[0];
```

```
int* linha1 = numeros[1];
```

```
int* linha2 = numeros[2];
```

```
int* linha3 = numeros[3];
```

```
int* linha4 = numeros[4];
```

Representação de Matrizes - Parte 4

Mas e se quiséssemos um novo ponteiro para apontar para a matriz como um todo? Como ela é um ponteiro que aponta para outros ponteiros de inteiro, a declaração é `int**`, ou seja, duas estrelas para representar "ponteiro de ponteiro":

```
int numeros[5][10];  
int** copia = numeros;
```

```
// as duas operações abaixo são idênticas  
numeros[0][0] = 10;  
copia[0][0] = 10;
```

main.c (ver 47) - Declarando e imprimindo matrizes

```
#include <stdio.h>

#define LINHAS    3
#define COLUNAS   3

int main() {
    // Declara uma matriz.
    int matriz[LINHAS][COLUNAS] = {1, 2, 3 , 4, 5, 6, 7, 8, 9};

    // Imprime a matriz.
    for (int i=0; i < LINHAS; i++) {
        for (int j=0; j < COLUNAS; j++) {
            printf("%d ", matriz[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

main.c (ver 48) - Lendo uma matriz

```
#include <stdio.h>
```

```
int main() {  
    int matriz[3][3];  
    for (int i = 0; i < 3; i++) {  
        for (int j = 0; j < 3; j++) {  
            printf("Digite o valor da posição [%d][%d]: ", i, j);  
            scanf("%d", &matriz[i][j]);  
        }  
    }  
    return 0;  
}
```

main.c (ver 49) - Passando matriz com número fixo de colunas

```
#include <stdio.h>

void imprimirMatriz(int matriz[][3], int linhas) {
    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", matriz[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int mat[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    imprimirMatriz(mat, 2);
    return 0;
}
```

main.c (ver 50) - Usando ponteiro simples com matriz tratada como vetor

```
#include <stdio.h>

void imprimirMatriz(int *matriz, int linhas, int colunas) {
    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < colunas; j++) {
            printf("%d ", matriz[i * colunas + j]);
        }
        printf("\n");
    }
}

int main() {
    int mat[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    imprimirMatriz(&mat[0][0], 2, 3);
    return 0;
}
```

Exercício 91 ~ 10min

Crie um programa em C que leia uma matriz 4 x 4 de números inteiros.

O programa deve:

- imprimir a matriz;
- calcular a soma de cada linha;
- calcular a soma de cada coluna;
- informar qual linha possui a maior soma;
- informar qual coluna possui a maior soma.

Exercício 92 ~ 10min

Crie um programa que leia uma matriz quadrada 5 x 5.

O programa deve:

- imprimir os elementos da diagonal principal;
- imprimir os elementos da diagonal secundária;
- calcular a soma da diagonal principal;
- calcular a soma da diagonal secundária;
- informar qual das duas diagonais possui a maior soma.

Exercício 93 ~ 10min

Crie um programa em C que leia uma matriz 3 x 4 e gere sua matriz transposta.

O programa deve:

- ler os valores da matriz original;
- armazenar a transposta em outra matriz;
- imprimir a matriz original;
- imprimir a matriz transposta.

Lembre-se:

- Matriz original: 3 x 4
- Matriz transposta: 4 x 3

Exercício 94 ~ 10min

Crie um programa que leia duas matrizes:

- $A = 2 \times 3$
- $B = 3 \times 2$

O programa deve calcular e imprimir a matriz resultante C , obtida pela multiplicação:

- $C = A \times B$

O cálculo de cada posição deve seguir a lógica:

- $C[i][j] = A[i][0] * B[0][j] + A[i][1] * B[1][j] + A[i][2] * B[2][j];$

Exercício 95 ~ 10min

Crie um programa em C que leia uma matriz quadrada 4 x 4.

O programa deve verificar se ela é uma matriz identidade.

Uma matriz identidade possui:

- 1 na diagonal principal
- 0 em todas as outras posições

Exemplo:

```
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
```

O programa deve imprimir:

A matriz é identidade.

ou

A matriz não é identidade.

Exercício 96 ~ 10min

Crie um programa que leia uma matriz quadrada 5 x 5.

O programa deve verificar se a matriz é simétrica.

Uma matriz é simétrica quando:

- $\text{matriz}[i][j] == \text{matriz}[j][i]$, para todas as posições.

Exemplo de matriz simétrica:

1	2	3
2	5	6
3	6	9

Exercício 97 ~ 10min

Crie um programa que leia uma matriz quadrada 6 x 6.

O programa deve encontrar:

- o maior valor da diagonal principal;
- o maior valor da diagonal secundária;
- o maior valor acima da diagonal principal;
- o maior valor abaixo da diagonal principal;
- o maior valor da matriz inteira.

Considere:

- acima da diagonal principal: posições em que coluna > linha
- abaixo da diagonal principal: posições em que linha > coluna

Exercício 98 ~ 10min

Crie um programa em C que leia uma matriz quadrada 4 x 4 e gere uma nova matriz correspondente à rotação da matriz original em 90 graus no sentido horário.

Exemplo:

Matriz original:

1	2	3
4	5	6
7	8	9

Matriz rotacionada:

7	4	1
8	5	2
9	6	3

O programa deve:

- ler a matriz original;
- criar uma matriz rotacionada;
- imprimir as duas matrizes.

Dica:

- $\text{rotacionada}[j][n - 1 - i] = \text{original}[i][j];$

Exercício 99 (Avançado)

Crie um programa em C que represente um tabuleiro de jogo da velha usando uma matriz 3 x 3 de caracteres.

O programa deve:

- inicializar a matriz com espaços vazios;
- permitir jogadas alternadas entre os jogadores X e O;
- impedir jogadas em posições já ocupadas;
- imprimir o tabuleiro após cada jogada;
- verificar vitória nas linhas;
- verificar vitória nas colunas;
- verificar vitória nas diagonais;
- verificar empate.

Desafio adicional: separar a lógica em funções, como:

- `void imprimirTabuleiro(char tabuleiro[3][3]);`
- `int verificarVitoria(char tabuleiro[3][3], char jogador);`
- `int verificarEmpate(char tabuleiro[3][3]);`

Exercício 100 (Avançado)

Crie um programa em C que leia uma matriz 5 x 5 contendo custos inteiros positivos.

Cada posição da matriz representa o custo para passar por aquela célula.

O objetivo é sair da posição:

[0][0]

e chegar até:

[4][4]

O jogador só pode se mover:

para a direita
para baixo

O programa deve calcular o menor custo total possível para chegar ao destino.

Exemplo:

1	3	1	2	9
2	1	8	1	3
4	2	1	5	1
6	1	2	1	2
9	3	4	1	1

O programa deve gerar uma matriz auxiliar com o menor custo acumulado até cada posição.

Desafio adicional: imprimir também o caminho escolhido.

AULA 13

(21/5/2026)

Prova 2